

IoT-based Smart Parking Management System (IoT-SPMS)

Submission #3 — Architectural & Detailed Design

Course: Software Engineering (SE252) **Project:** Smart Parking System for University Campus — HCMUT
Document status: v1.0 (builds on Submissions #1 and #2)

Table of Contents

1. [Architectural Overview](#)
 2. [Development / Implementation View \(Components\)](#)
 3. [Deployment View](#)
 4. [Domain Class Diagram](#)
 5. [Design Patterns Applied](#)
 6. [Class & Method Descriptions](#)
 7. [Test Cases](#)
-

1. Architectural Overview

1.1 Architectural style

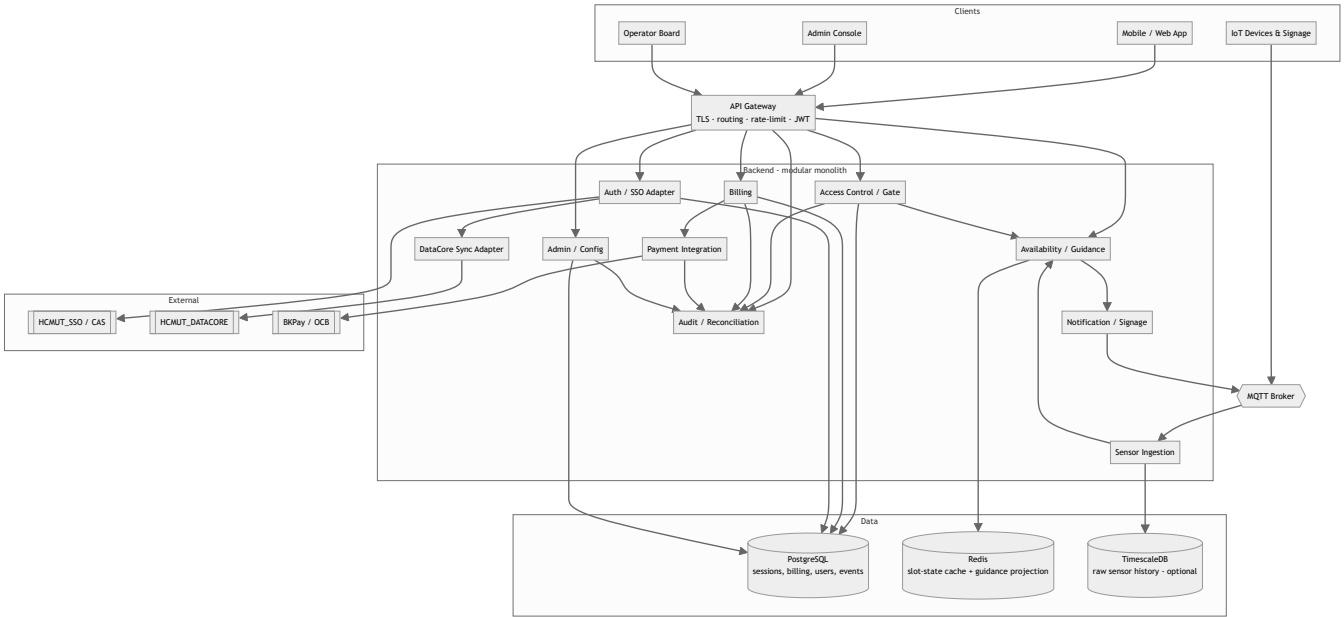
The system adopts a **layered, event-driven, service-oriented architecture** deployed as a **modular monolith** for the MVP (ten logical services as internal modules behind one API gateway, one MQTT broker, one PostgreSQL database). This preserves clean service boundaries and the story of a future microservice split, while remaining buildable and demonstrable by a student team. Only the **Sensor Ingestion** and **Payment** modules are candidates for a first physical split, because they own the two hardest failure surfaces (noisy IoT traffic; slow/flaky external gateway).

Rationale for the decomposition: services are split by **rate-of-change** and **data ownership**, not by CRUD entity. Hardware protocols churn (→ isolate in Ingestion and Gate); money rules and gateways change independently (→ isolate in Billing and Payment); read-heavy availability scales separately from writes (→ CQRS-lite Availability projection).

1.2 Quality-driven decisions (traceable to NFRs)

Decision	Serves NFR
MQTT broker as the single IoT→backend seam with retained messages + QoS1	NFR-PERF-02, NFR-REL-03
Edge/gateway store-and-forward + offline gate decisions	NFR-REL-02
Circuit Breaker + fallback on every external call (SSO/DATACORE/BKPay)	NFR-REL-01, NFR-COMP-01
CQRS-lite Availability read projection in Redis	NFR-PERF-02, NFR-SCAL-01
Hash-chained, append-only audit log	NFR-SEC-03
Idempotent IPN handler + daily reconciliation	NFR-SEC-04, FR-ADM-06
RBAC gate + object-level ABAC middleware	NFR-SEC-02

2. Development / Implementation View (Components)

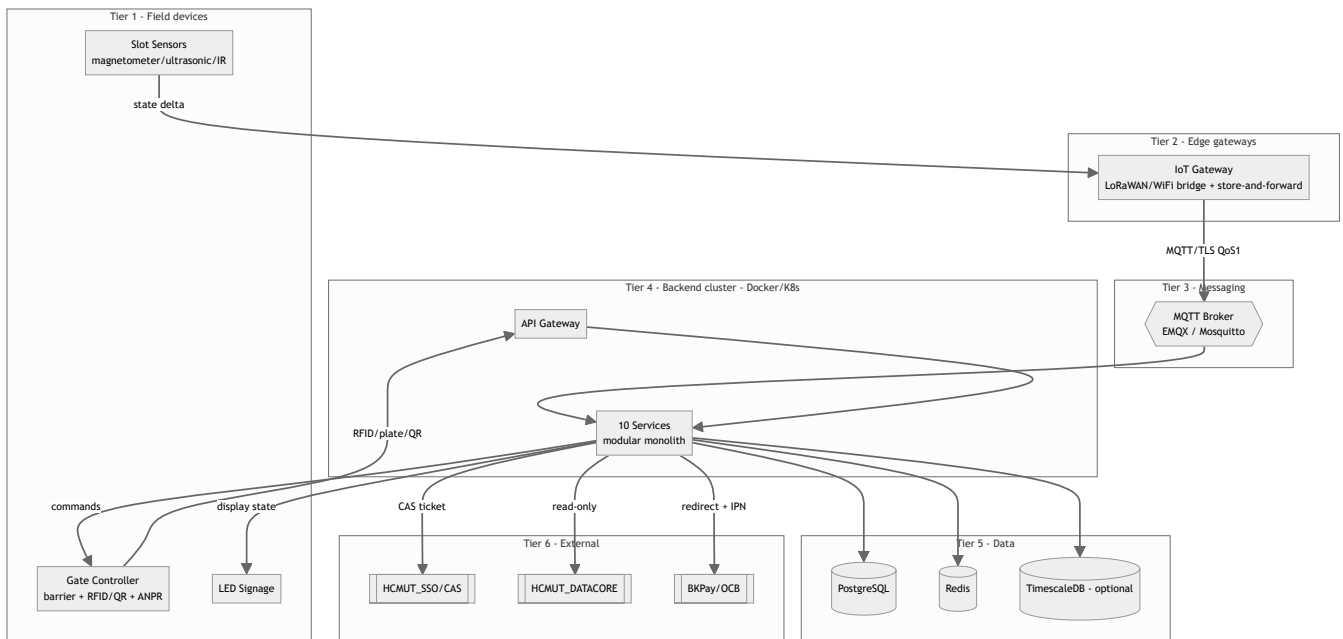


Module responsibilities

#	Module	Responsibility	Owns
1	API Gateway	Single ingress; TLS, routing, rate-limit, JWT validation, CORS.	—
2	Auth / SSO Adapter	CAS login dance; mints internal JWT; maps SSO identity → local User/Role.	session/token
3	Access Control / Gate	Entry/exit decisions; plate/RFID/QR match; barrier commands; writes AccessEvent.	AccessEvent
4	Sensor Ingestion	Only MQTT consumer that writes slot state; validate/dedupe/debounce; emit SlotStateChanged.	Slot/Sensor state
5	Availability / Guidance	Read-optimized free-count projection per Zone; nearest-free routing; feeds signage/app.	availability projection
6	Billing	ParkingSession pricing lifecycle; applies PricingPolicy; produces BillingRecord.	Session, BillingRecord
7	Payment Integration	BKPay adapter (redirect + IPN); idempotency + retries; reconciles Payment vs BillingRecord.	Payment
8	DataCore Sync Adapter	Read-only pull of identity/vehicle data; local cache with TTL.	user cache
9	Admin / Config	Zones, slots, sensors, pricing policies, device registry, feature flags.	config
10	Notification / Signage	Fan-out to LED signage, mobile push, SMS/email.	delivery
—	Audit / Reconciliation	Hash-chained audit log; daily settlement reconciliation.	audit log

3. Deployment View

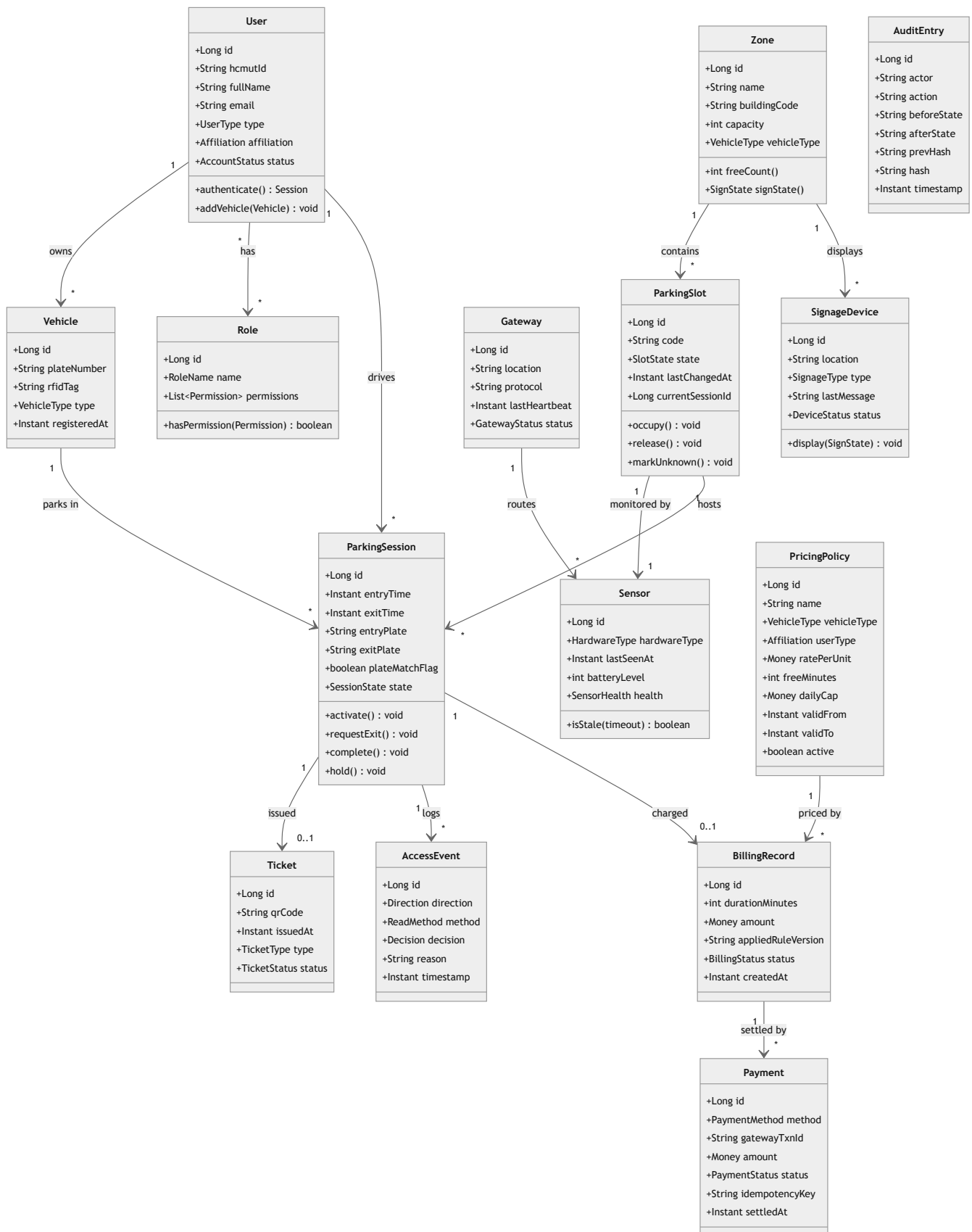
Six tiers, bottom-up. Device↔broker and service↔external links use mutual TLS where possible.



Node notes

- Backend services are stateless & horizontally scalable, **except Sensor Ingestion** which is partitioned by gateway/zone (ordering matters per slot).
- Redis holds the slot-state cache and the guidance projection (CQRS read side); PostgreSQL is the transactional write model.
- Gateways buffer messages during backhaul outages and replay in order on reconnect (store-and-forward), satisfying NFR-REL-02.

4. Domain Class Diagram

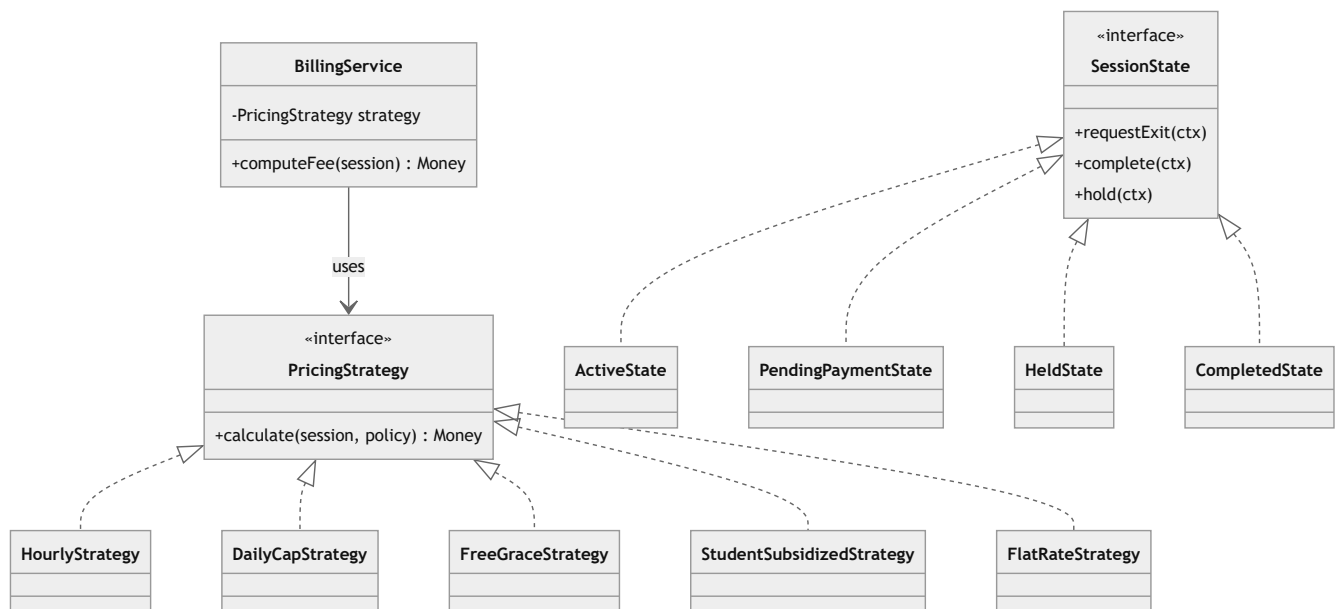


Class stereotypes (for grading clarity):

- **Entity:** User, Role, Vehicle, Zone, ParkingSlot, Sensor, Gateway, ParkingSession, Ticket, AccessEvent, PricingPolicy, BillingRecord, Payment, SignageDevice, AuditEntry.
- **Boundary/View:** LoginView, AvailabilityMapView, OperatorBoardView, ExitTerminalView, AdminTariffView.
- **Control/Service:** AuthService, AccessControlService, SensorIngestionService, AvailabilityService, BillingService, PaymentService, AdminService, NotificationService, AuditService (see §6).

5. Design Patterns Applied

Pattern	Where	Why
Observer	<code>SensorIngestion</code> (Subject) → <code>Availability</code> , <code>Notification</code> , <code>AccessControl</code> (Observers) via <code>SlotStateChangedEvent</code>	Decouples "who knows a slot freed" from "who reacts"; implemented as MQTT fan-out + in-app event bus.
State	<code>ParkingSession</code> and <code>ParkingSlot</code> lifecycles as State classes	Legal transitions only; prevents "paid before exit" bugs; testable state machine (see ST-1/ST-2 in Submission #2).
Strategy	<code>PricingStrategy</code> implementations: <code>HourlyStrategy</code> , <code>DailyCapStrategy</code> , <code>FreeGraceStrategy</code> , <code>StudentSubsidizedStrategy</code> , <code>FlatRateStrategy</code>	New rate schemes = new class, no edit to Billing core.
Adapter	<code>SsoAdapter</code> , <code>DataCoreAdapter</code> , <code>PaymentGatewayAdapter</code> → <code>BkPayAdapter</code>	Clean internal port; swap BKPay / mock SSO in tests without touching domain.
Circuit Breaker	Wrap outbound SSO/DATACORE/BKPay calls (+ bulkhead, timeout, retry-with-backoff)	Documented fallback per external system → "what happens when BKPay is down" has an answer.
Repository	One per aggregate (<code>SessionRepository</code> , <code>SlotRepository</code> , ...)	Isolates persistence from domain logic.
CQRS-lite	Availability reads a denormalized Redis projection separate from the Postgres write model	Read scaling without touching writes (NFR-SCAL-01).



6. Class & Method Descriptions

Every method in the design carries a description of purpose, parameters, return, and key behaviour/exceptions. (Control/service classes shown; entity getters/setters omitted for brevity.)

6.1 AuthService (control — Auth/SSO Adapter)

Method	Description
<code>String buildLoginRedirect(String serviceUrl)</code>	Returns the CAS login redirect URL (<code>/cas/login?service=...</code>) for an unauthenticated request. <code>serviceUrl</code> must byte-match the later validation call.
<code>AuthResult validateTicket(String serviceTicket, String serviceUrl)</code>	Server-to-server call to <code>/p3/serviceValidate</code> ; on <code>authenticationSuccess</code> extracts uid + attributes and returns the identity. Throws <code>TicketValidationException</code> on failure/replay.
<code>Session mintSession(AuthResult r)</code>	JIT-provisions a local <code>User</code> if new, enriches from DataCore, and issues a signed JWT/local session.
<code>void logout(String sessionIndex)</code>	Handles CAS single-logout (fire-and-forget): maps SessionIndex → local session and invalidates it.

6.2 `AccessControlService` (control — Gate)

Method	Description
<code>AccessDecision handleEntry(GateRead read)</code>	Validates credential + blacklist + zone availability; creates an ACTIVE <code>ParkingSession</code> ; writes <code>AccessEvent(GRANTED)</code> ; decrements free-count. Returns GRANTED/DENIED + reason.
<code>AccessDecision handleExit(GateRead read)</code>	Finds the active session; calls <code>verifyPlateMatch</code> ; triggers billing/payment; on success completes the session and increments free-count.
<code>boolean verifyPlateMatch(ParkingSession s, String exitPlate)</code>	Compares entry vs exit plate; on mismatch raises <code>PLATE_MISMATCH</code> alarm, holds the session, and returns false (FR-EXT-04).
<code>Ticket startVisitorSession(String plate)</code>	Opens a plate-only session and issues a QR ticket for the visitor/temporary-access flow (FR-ENT-05).
<code>void commandBarrier(GateId g, BarrierAction a)</code>	Sends open/close to the barrier controller with the fail-safe interlock (NFR-SAFE-01).

6.3 `SensorIngestionService` (control — the only slot-state writer)

Method	Description
<code>void onMessage(MqttMessage m)</code>	Entry point for every MQTT slot message: debounce, dedupe, validate, then apply. Drops invalid payloads and increments an error metric.
<code>void applyStateChange(Long slotId, SlotState s, Instant ts)</code>	Updates authoritative slot state + <code>lastSeenAt</code> ; publishes a <code>SlotStateChangedEvent</code> (Observer subject).
<code>void runStalenessWatchdog()</code>	Periodic job: any slot older than the staleness timeout → <code>markUnknown()</code> + alert; never counts unknown as free (FR-OCC-05).
<code>void reconcile(Zone z)</code>	Periodically compares sensor-derived state vs gate entry/exit counts and flags discrepancies (FR-OCC-04).

6.4 `AvailabilityService` (control — read projection / Observer)

Method	Description
<code>void onSlotStateChanged(SlotStateChangedEvent e)</code>	Applies the delta to the per-zone free-count in the Redis projection.
<code>int freeCount(Long zoneId)</code>	Returns the current free count for a zone (excludes UNKNOWN/OUT_OF_SERVICE).
<code>SignState signState(Long zoneId)</code>	Maps occupancy % to a sign state using the admin-configured thresholds (FR-SIG-01/04).
<code>Zone nearestFreeZone(Long fromZoneId)</code>	Returns the nearest non-full zone for directional guidance (FR-SIG-03).

6.5 BillingService (control)

Method	Description
<code>PricingPolicy</code> <code>resolvePolicy(ParkingSession s)</code>	Selects the active <code>PricingPolicy</code> for the session's vehicle type + affiliation at the session's time (temporal lookup).
<code>Money</code> <code>computeFee(ParkingSession s)</code>	Uses the chosen <code>PricingStrategy</code> to compute the fee (duration, free minutes, daily cap, rounding).
<code>BillingRecord</code> <code>closeSession(ParkingSession s)</code>	Freezes amount + rule version onto a new <code>BillingRecord</code> ; transitions the session toward COMPLETED (FR-EXT-06).
<code>Statement</code> <code>generatePeriodStatement(User u, Period p)</code>	Aggregates charge lines over a billing period into a statement and initiates a BkPay request (FR-EXT-08).

6.6 PaymentService (control — Payment Integration)

Method	Description
<code>PaymentRequest</code> <code>initiate(BillingRecord b, PaymentMethod m)</code>	Creates a PENDING <code>Payment</code> with its own order/reference id (the correlation key); builds the signed redirect request.
<code>void handleIpn(IpnCallback cb)</code>	Verifies HMAC first; checks the transaction + amount; conditional update <code>PENDING→SETTLED WHERE status='PENDING'</code> (idempotent); rejects stale/replayed callbacks.
<code>boolean isDuplicate(String eventId)</code>	Short-circuits already-processed callbacks (idempotency store).
<code>void refund(Payment p, Money amount)</code>	Issues a refund and records an audit + reconciliation entry.

6.7 AdminService / NotificationService / AuditService

Method	Description
<code>PricingPolicy</code> <code>upsertPolicy(PolicyDraft d)</code>	Validates non-overlapping active rules; versions with <code>validFrom/validTo</code> ; audits the change (FR-ADM-01).
<code>void assignRole(User u, Role r)</code>	Grants/revokes roles (data-driven RBAC) with audit (FR-ADM-02).
<code>void pushSignage(Zone z, SignState s)</code>	<code>NotificationService</code> : fans out the sign state to signage devices (FR-SIG-02/03).
<code>AuditEntry</code> <code>append(String actor, String action, Object before, Object after)</code>	<code>AuditService</code> : writes a hash-chained, append-only audit row <code>hash = H(prevHash + rowData)</code> (NFR-SEC-03).
<code>List<ReconBreak></code> <code>reconcile(Date day)</code>	Matches local settled payments against the bank settlement file; records unmatched items as reconciliation breaks (FR-ADM-06).

7. Test Cases (*bonus*)

Representative cases spanning the highest-risk flows (happy path + the failure/edge flows where rubric points live).

TC-ID	Requirement	Precondition	Steps	Expected result
TC-01	FR-ENT-04	Valid member, zone not full	Present card + plate at entry	ACTIVE session created; free-count – 1; GRANTED logged
TC-02	FR-ENT-06	Zone full, no alternative	Present card at entry	Entry DENIED; sign shows FULL; DENIED reason logged
TC-03	FR-ENT-05	No credential	Vehicle at lane	Visitor plate-only session + QR ticket issued
TC-04	FR-EXT-03	Student, motorbike, 5h33m stay	Exit & compute	Fee applies student – 15% + daily-cap + rounding correctly
TC-05	FR-EXT-04	Entry plate ≠ exit plate	Present card at exit	Anti-theft alarm raised; session HELD; barrier blocked; operator notified
TC-06	FR-EXT-06	Session closing; rate later changes	Close then change tariff	Historical bill unchanged (amount + rule version frozen)
TC-07	FR-OCC-05	Sensor silent > staleness timeout	Wait past timeout	Slot → UNKNOWN; alert ≤30s; not counted as free
TC-08	FR-SIG-01/04	Occupancy crosses 90%	Fill zone to 90%	Sign state → ORANGE "NEARLY FULL"
TC-09	Payment idempotency	IPN delivered twice	Send duplicate IPN	Second IPN is a no-op; single SETTLED payment
TC-10	FR-ADM-06	Local SETTLED, no settlement line	Run reconciliation	Item flagged as reconciliation break
TC-11	NFR-REL-02	Backend offline	Entry during outage	Local decision cached; event synced ≤60s on reconnect
TC-12	NFR-SEC-02	end_user calls another user's session	<code>GET /sessions/{id}</code> not owned	403 Forbidden (object-level ABAC)
TC-13	FR-AUD-04	Any privileged action	Admin changes a tariff	Hash-chained audit row written; chain verifies
TC-14	NFR-SAFE-01	Vehicle under barrier	Command close	Barrier does not close; hazard warning